

Ex. No: 8

Exercise 1

TF-IDF Vector Embedding

Aim

To implement Term Frequency-Inverse Document Frequency (TF-IDF) vector embeddings for small and large corpus datasets to represent documents as numerical vectors capturing the relative importance of terms.

Procedure/Pseudocode

- 1. Load and preprocess the corpus (tokenize, lowercase, remove stop words and punctuation).
- 2. Term Frequency (TF): $TF(t, d) = \text{count}(t \text{ in } d) / \text{total_terms}(d)$.
- 3. Inverse Document Frequency (IDF): $IDF(t) = \log(N / df(t))$ where N = total documents, $df(t)$ = docs containing t .
- 4. TF-IDF score: $TF-IDF(t, d) = TF(t, d) \times IDF(t)$.
- 5. Build a document-term TF-IDF matrix where rows = documents, columns = vocabulary terms.
- 6. Apply L2 normalization on the document vectors.
- 7. For large corpus: use sparse matrix representation (scipy.sparse) for memory efficiency.
- 8. Retrieve top-N terms per document by TF-IDF score to interpret the embedding.

Libraries Used

- 1. sklearn.feature_extraction.text.TfidfVectorizer – for TF-IDF computation
- 2. numpy – for matrix operations
- 3. scipy.sparse – for large corpus sparse matrices
- 4. nltk – for preprocessing
- 5. pandas – for result display

Test Cases

- 1. Document 1: 'Machine learning is amazing' → TF-IDF assigns low score to 'is', high to 'machine'
- 2. Document 2: 'Deep learning neural networks' → 'neural' gets high IDF (rare across docs)
- 3. Common word test: 'the' appears in all docs → $IDF \approx 0 \rightarrow TF-IDF \approx 0$

- 4. Large corpus: sparse TF-IDF matrix for 18,000 newsgroup docs with vocab size $\approx 100,000$

Results

TF-IDF vectors successfully captured term importance relative to the corpus. Rare but informative terms received high TF-IDF scores. The method outperformed raw term frequency for document retrieval and classification tasks. Sparse matrix representation made computation feasible for large corpora.

Exercise 2 Cosine Similarity Measure for all Set of Words

Aim

To implement Cosine Similarity measure to compute pairwise semantic similarity between all word vectors in small and large corpus datasets.

Procedure/Pseudocode

- 1. Obtain word vector representations (TF-IDF or word embeddings) for all words in the vocabulary.
- 2. Cosine Similarity Formula: $\cos(A, B) = (A \cdot B) / (\|A\| \times \|B\|)$.
- 3. Compute cosine similarity for all pairs of word vectors.
- 4. For small corpus: compute full pairwise similarity matrix ($|V| \times |V|$).
- 5. For large corpus: use approximate nearest neighbor methods (Annoy or FAISS) to avoid $O(|V|^2)$ computation.
- 6. Identify top-K most similar words for each query word.
- 7. Visualize similarity clusters using heatmaps for small vocabulary subsets.

Libraries Used

- 1. numpy – for dot product and norm computation
- 2. sklearn.metrics.pairwise.cosine_similarity – for efficient batch computation
- 3. scipy.spatial.distance – for cosine distance
- 4. faiss – for approximate nearest neighbor on large vocab
- 5. matplotlib/seaborn – for similarity heatmap visualization

Test Cases

- 1. $\cos(\text{'king'}, \text{'queen'}) \approx 0.85$ (high similarity – royalty domain)

- 2. $\cos(\text{'king'}, \text{'table'}) \approx 0.12$ (low similarity – unrelated)
- 3. $\cos(\text{'cat'}, \text{'cat'}) = 1.0$ (self-similarity = 1)
- 4. Top-5 similar words to 'bank': ['financial', 'credit', 'loan', 'deposit', 'interest']

Results

Cosine similarity successfully identified semantically related word pairs. Words from the same domain consistently scored high similarity (>0.75). The approximate nearest neighbor approach (FAISS) reduced large corpus query time from $O(N)$ to $O(\log N)$, making real-time similarity lookup feasible.

Exercise 3 Pointwise Mutual Information (PMI) Score for all Set of Words

Aim

To implement Pointwise Mutual Information (PMI) score to measure the statistical association between all pairs of words in small and large corpus datasets, identifying collocations and semantically related word pairs.

Procedure/Pseudocode

- 1. Load and preprocess corpus; build co-occurrence counts within a context window (e.g., window size 5).
- 2. Compute unigram probabilities $P(w) = \text{count}(w) / \text{total_tokens}$ for each word.
- 3. Compute co-occurrence probabilities $P(w_1, w_2) = \text{count}(w_1, w_2) / \text{total_bigrams}$.
- 4. PMI Formula: $\text{PMI}(w_1, w_2) = \log_2 [P(w_1, w_2) / (P(w_1) \times P(w_2))]$.
- 5. Positive PMI (PPMI): $\text{PPMI}(w_1, w_2) = \max(0, \text{PMI}(w_1, w_2))$ to remove negative associations.
- 6. Build a PPMI word-context matrix for all word pairs in vocabulary.
- 7. For large corpus: use sparse matrix representation and threshold minimum co-occurrence count.
- 8. Rank word pairs by PPMI to extract collocations and word associations.

Libraries Used

- 1. numpy – for probability and log computation
- 2. scipy.sparse – for large PPMI matrices
- 3. collections.Counter – for co-occurrence counting
- 4. pandas – for result tabulation and ranking
- 5. nltk – for corpus loading and bigram extraction

Test Cases

- 1. $\text{PMI}(\text{'New'}, \text{'York'}) = \text{high positive value}$ (strong collocation)
- 2. $\text{PMI}(\text{'the'}, \text{'a'}) \approx 0$ or negative (high frequency, low mutual information)
- 3. $\text{PPMI}(\text{'neural'}, \text{'network'}) > \text{PPMI}(\text{'neural'}, \text{'apple'})$ confirming domain association
- 4. Sparse matrix test: Large corpus $\rightarrow$ most word pairs have co-occurrence = 0 (sparse)

Results

PMI scores successfully identified strong word associations and collocations in all corpora. PPMI removed spurious negative associations, making the matrix suitable for downstream NLP tasks. Named entity collocations (New York, United States) scored highest. The PPMI matrix improved word similarity tasks compared to raw co-occurrence counts.